

MLP Classifier

Многослойный перцептрон
для задачи классификации

Сагун Александр
Житков Никита

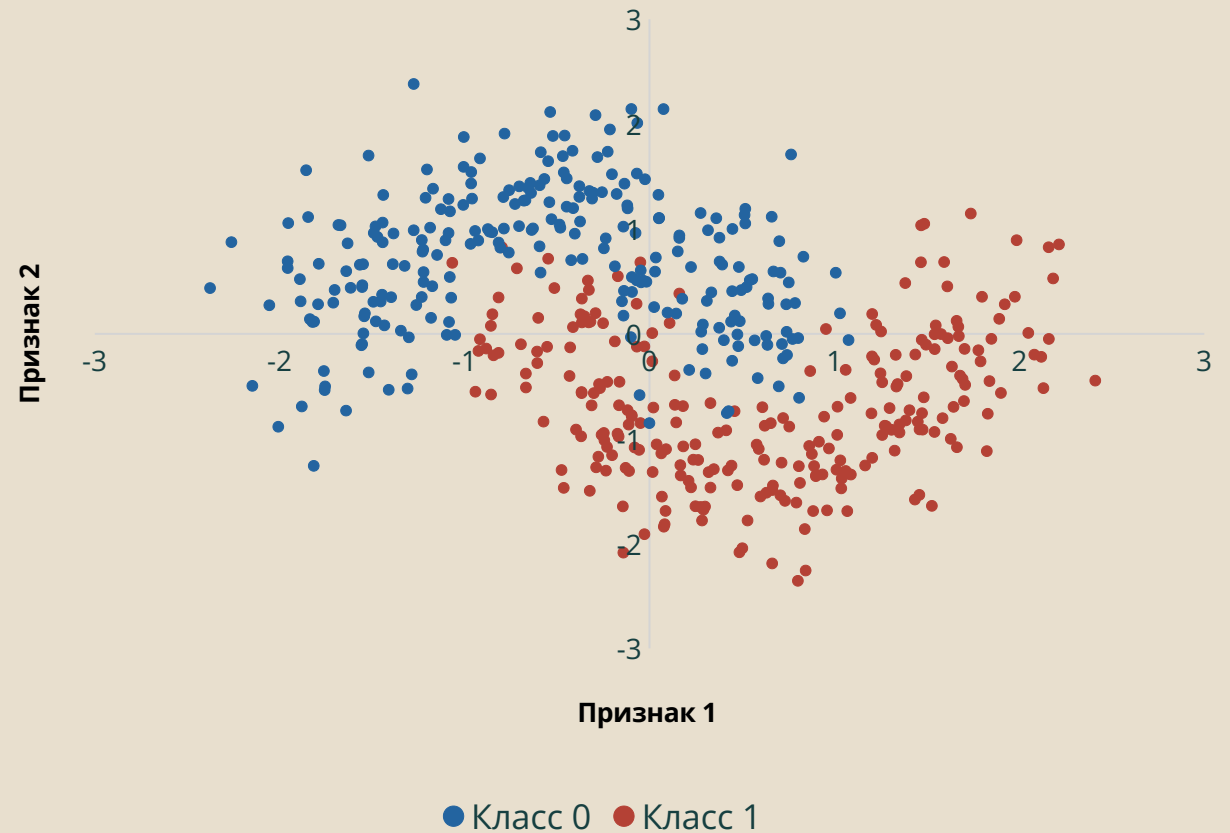
Зачем нужен MLP Classifier

Классификация

По признакам объекта X
предсказываем его класс y .

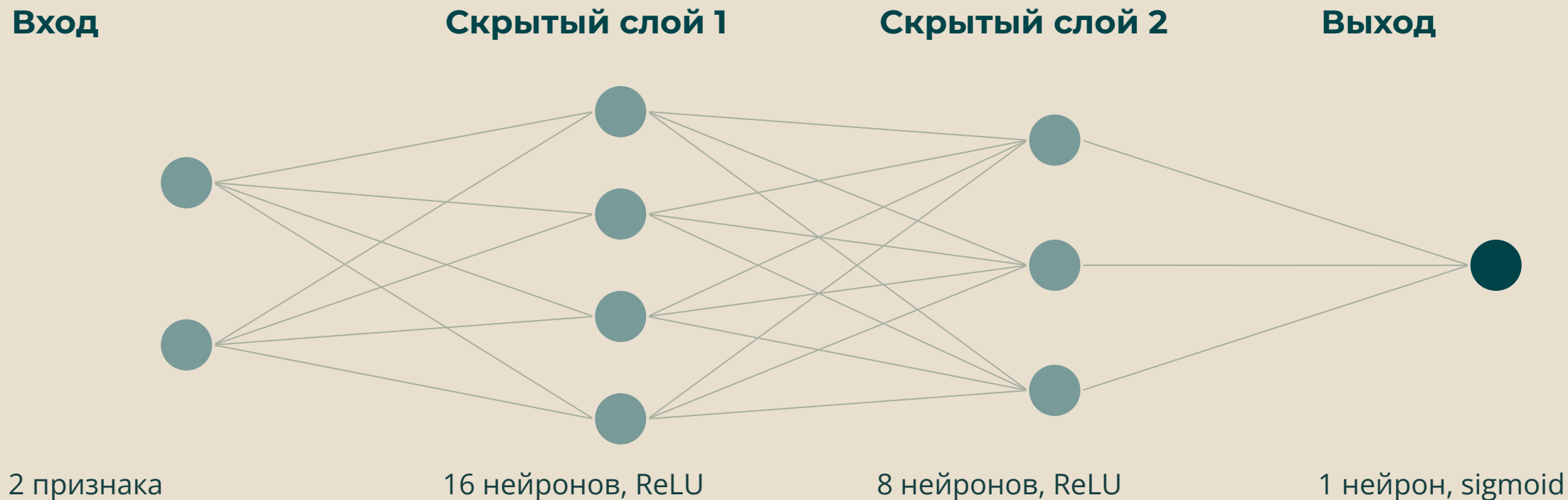
Примеры
Спам / не спам
Болен / здоров

Нелинейные активации позволяют
строить сложные границы классов.



make_moons: 500 объектов, 2 признака, 2 класса

Архитектура MLP



$$z = Wx + b$$

$$a = f(z)$$

Бинарный выход: sigmoid
Многоклассовый выход: softmax

Как обучается MLP

01

Прямой проход

Вход X , предсказание $\hat{y}$, расчёт cross-entropy.

02

Обратный проход

Правило цепочки даёт градиенты по весам.

03

Обновление весов

Оптимизатор меняет веса. Цикл повторяется по эпохам.

$$W \leftarrow W - \eta \cdot \partial L / \partial W$$

η — шаг обучения
SGD: базовое обновление
Adam: адаптивные шаги

MLPClassifier в scikit-learn

Базовая конфигурация

```
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation='relu',
    solver='adam',
    alpha=0.0001,
    max_iter=500,
    early_stopping=True,
    random_state=42
)
```

Основные параметры

hidden_layer_sizes
Число слоёв и нейронов

activation / solver
Активация / оптимизатор

alpha
Сила L2-регуляризации

early_stopping
Остановка по валидации

learning_rate_init задаёт начальный шаг обучения. random_state фиксирует случайность.

Python-пример: подготовка данных

demo_mlp.py · подготовка данных

```
RANDOM_STATE = 42
X, y = make_moons(
    n_samples=500, noise=0.25,
    random_state=RANDOM_STATE
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y,
    random_state=RANDOM_STATE
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

350

объектов train

150

объектов test

75 + 75

объектов каждого
класса в test

StandardScaler обучается только на train. К test применяем уже найденные средние и масштабы.

Python-пример: обучение MLP

demo_mlp.py · обучение и предсказание

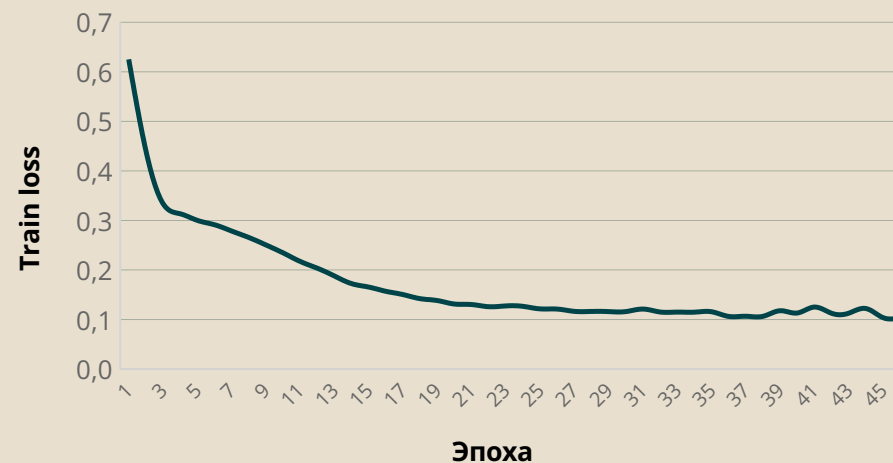
```
mlp = MLPClassifier(  
    hidden_layer_sizes=(16, 8),  
    activation='relu', solver='adam',  
    alpha=0.0001, batch_size=32,  
    learning_rate_init=0.01,  
    max_iter=500, early_stopping=True,  
    n_iter_no_change=20,  
    random_state=RANDOM_STATE,  
    verbose=False  
)  
mlp.fit(X_train_scaled, y_train)  
y_pred = mlp.predict(X_test_scaled)
```

```
coefs_: (2, 16), (16, 8), (8, 1)  
intercepts_: (16,), (8,), (1,)
```

95,33%

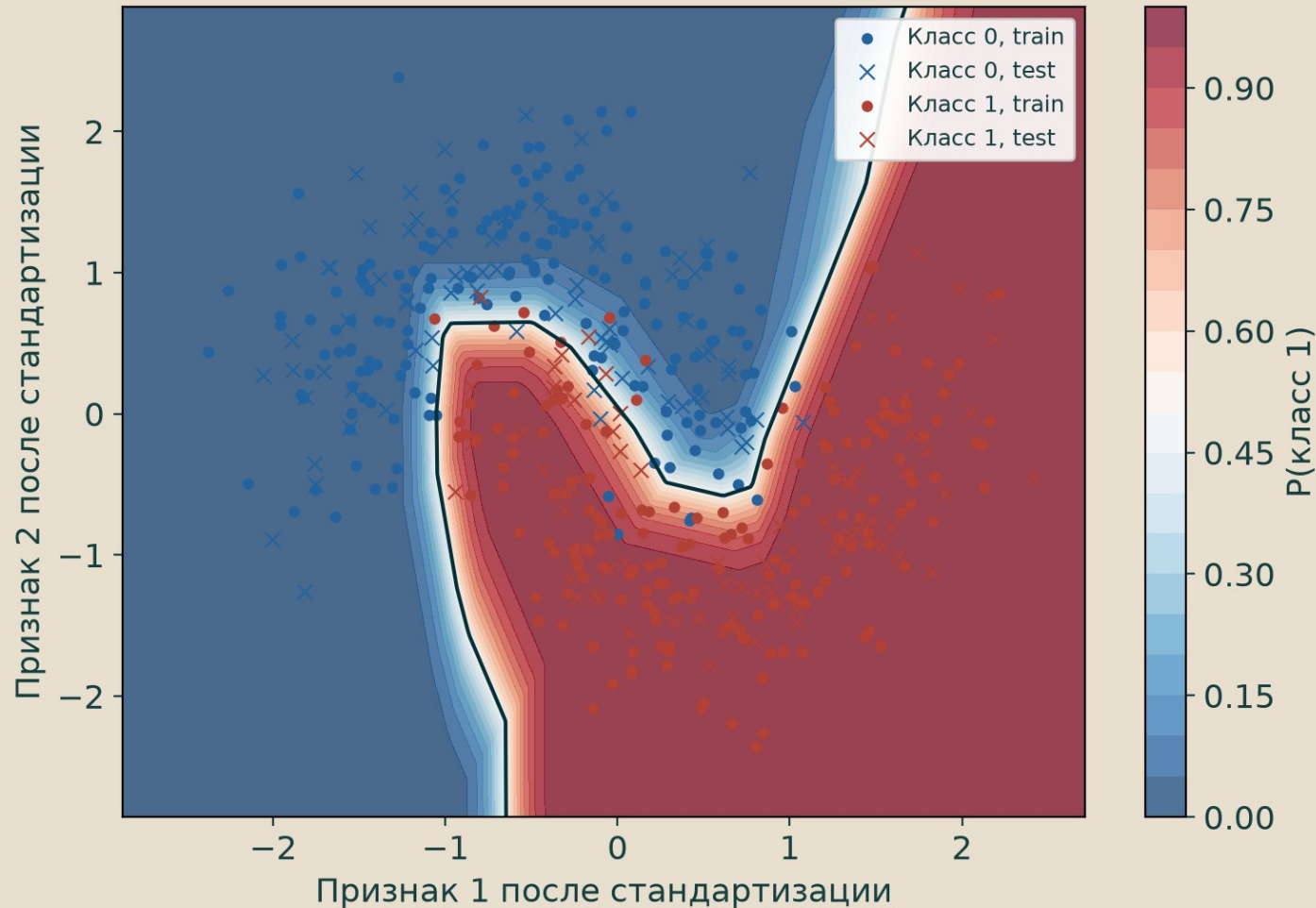
точность на test: 143 из 150

46 эпох **loss = 0,10336**



Ранняя остановка отслеживает validation accuracy.
Она не гарантирует отсутствие переобучения.

Python-пример: граница решения



demo_mlp.py · фрагмент визуализации

```
grid = np.c_[xx.ravel(), yy.ravel()]
Z = mlp.predict_proba(grid)[:, 1]
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z,
              levels=20, cmap='RdBu_r')
```

Нелинейная граница

Синий фон: класс 0
Красный фон: класс 1
Линия: $P(\text{класс 1}) = 0,5$
Крестики: тестовые объекты

7 ошибок на 150 объектах.
Precision и recall обоих классов
близки: 0,95–0,96.

Численный пример: постановка

Сеть 2 → 2 → 1

$x = [0.5, -0.2]$
 $y = 1$

Активация всех нейронов: sigmoid
Функция потерь: binary cross-entropy
Шаг обучения $\eta = 0.5$

$$\sigma(z) = 1 / (1 + \exp(-z))$$

numerical_mlp.py · начальные значения

```
x = np.array([0.5, -0.2])  
y = 1.0
```

```
W1 = np.array([[0.1, 0.2],  
               [0.3, -0.4]])
```

```
b1 = np.array([0.05, -0.1])
```

```
W2 = np.array([[0.6, -0.5]])
```

```
b2 = np.array([0.2])
```

Численный пример: forward pass

numerical_mlp.py · прямой проход

```
def sigmoid(x):  
    return 1.0 / (1.0 + np.exp(-x))  
  
z1 = W1 @ x + b1  
a1 = sigmoid(z1)  
z2 = W2 @ a1 + b2  
a2 = sigmoid(z2)  
loss = -(y * np.log(a2)  
         + (1 - y) * np.log(1 - a2))
```

Результат вычислений

$z_1 = [0.06000, 0.13000]$
 $a_1 = [0.51500, 0.53245]$
 $z_2 = 0.24277$

0,56040

вероятность класса 1

$$L = -\ln(0,56040) = 0,57911$$

Целевой ответ $y = 1$. Обновление весов должно увеличить вероятность этого класса.

Backward pass и обновление весов

$$\partial L / \partial a_2 = (a_2 - y) / [a_2(1 - a_2)] \quad \partial a_2 / \partial z_2 = a_2(1 - a_2)$$

По правилу цепочки: $\partial L / \partial z_2 = a_2 - y = -0,439604$

numerical_mlp.py · градиенты и шаг

```
dz2 = a2 - y
dW2 = dz2.reshape(1, 1) @ a1.reshape(1, -1)
db2 = dz2
da1 = (W2.T @ dz2.reshape(1, 1)).ravel()
dz1 = da1 * a1 * (1 - a1)
dW1 = np.outer(dz1, x)
db1 = dz1
lr = 0.5
W2_new = W2 - lr * dW2
b2_new = b2 - lr * db2
W1_new = W1 - lr * dW1
b1_new = b1 - lr * db1
```

Новые параметры

$W_1 = \begin{bmatrix} 0.11647, & 0.19341 \\ 0.28632, & -0.39453 \end{bmatrix}$
 $b_1 = [0.08294, -0.12736]$

$W_2 = [[0.71320, -0.38297]]$
 $b_2 = [0.41980]$

Вероятность: 0,56040 → 0,64431
Потеря: 0,57911 → 0,43958

Альтернативные варианты

Модель	Сильная сторона	Что учитывать
Logistic Regression	Простая линейная база	Не моделирует нелинейную границу сама по себе
SVM с RBF-ядром	Нелинейные границы	Масштабирование признаков, стоимость обучения
KNN	Простой локальный метод	Чувствительность к масштабу, шуму и выбору k
Random Forest	Ансамбль деревьев	Размер ансамбля и глубина деревьев
Gradient Boosting	Последовательное улучшение деревьев	Настройка глубины, шага и числа деревьев
MLP	Гибкая архитектура сети	Масштабирование и подбор гиперпараметров

Качество зависит от данных и настроек. Одного эксперимента недостаточно для общего рейтинга моделей.

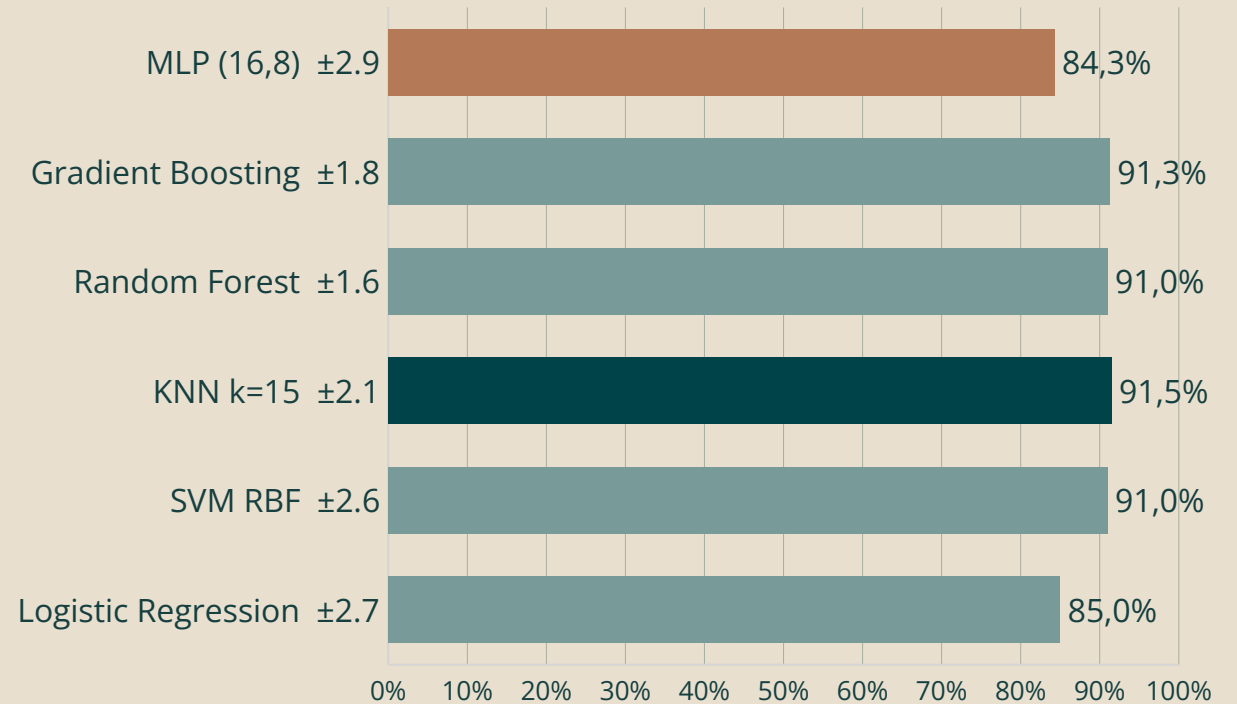
Сравнение на Python

make_moons: 1000 объектов, noise = 0,3. Стратифицированная кросс-валидация, 5 разбиений.

compare_models.py · оценка моделей

```
cv = StratifiedKFold(
    n_splits=5, shuffle=True,
    random_state=42
)
for name, model in models.items():
    scores = cross_val_score(
        model, X, y, cv=cv,
        scoring='accuracy'
    )
    print(name, scores.mean(),
          scores.std())
```

MLP: 84,3%. Здесь шаг 0,001 и batch_size = auto.
Данные и оценка отличаются от примера с 95,33%.



Подписи ±: стандартное отклонение ассигасу
между 5 разбиениями, в процентных пунктах.

Итоги

**MLP строит нелинейные границы
и поддерживает несколько классов.**

Сильные стороны

- Нелинейные зависимости
- Гибкая архитектура
- Бинарная и многоклассовая классификация

Условия хорошего результата

- Масштабирование признаков
- Подбор гиперпараметров
- Контроль переобучения и затрат на обучение

Модель выбираем по проверке на данных задачи.

Спасибо за внимание